FitWise App: Code Structure and Viva Preparation Guide

Oct 25, 2026 · @Sarvesh

1. Overview

FitWise is a trainer supervised workout planning system: a machine learning model suggests a workout category, two generators build a weekly plan from it, a validator checks each plan, and a human trainer must approve a plan before the member ever sees it. This guide explains how the code is organised and why, so you can answer a reviewer's questions about any part of it.

The system has four parts that talk to each other over HTTP:

Part	Technology	What it does	Where it lives
Web app and API	Next.js 16 (App Router), React 19, TypeScript	Admin, trainer and member portals, plus all the server API routes	FitWise_App/web
Database and auth	Supabase (PostgreSQL, Supabase Auth, Row Level Security)	Stores users, members, assessments, predictions, plans, completions; handles login	FitWise_App/web/supabase/migrations
ML service	Python, FastAPI, scikit learn (Logistic Regression)	Predicts Cardio, Strength, HIIT or Yoga from 8 body and experience features	FitWise_Prototype_Lab/ml

Part	Technology	What it does	Where it lives
Mobile app	Flutter (Dart), supabase_flutter	Member facing app: see the approved plan, log workouts, view progress	FitWise_App/mobile

The AI part uses Google Gemini (model set by GEMINI_MODEL, default gemini-3.6-flash) through the @google/genai SDK, called only from the server.

Three user roles exist: **admin** (manages trainers, the exercise catalogue and member assignment), **trainer** (creates members, records assessments, runs predictions, generates and approves plans) and **member** (follows the approved plan on web or mobile).

2. Code map: where each feature lives

When the reviewer says "show me the code that does X", find X in the first column and open the file. Paths start from FitWise_App/web unless they say otherwise. Line numbers are from the current code.

Feature	What the user clicks	Front end file (sends the request)	Back end file	Function, lines
Login (web)	Sign In on /login	app/login/page.tsx	Supabase Auth	signInWithPassword
Keep login alive	every page load	none	proxy.ts	proxy(), 8 to 42
Block wrong role from a page	opening /trainer or /admin	app/trainer/layout.tsx, app/admin/layout.tsx	lib/auth.ts	verifySession() 19 to 36, requireTrainer() 47 to 51, requireAdmin() 53 to 57
Block wrong role from an API	any API call	none	lib/authApi.ts	requireRoleApi() 8 to 24
Trainer signs up	Apply on /signup	app/signup/page.tsx	app/api/auth/trainer-signup/route.ts	POST
Admin approves or deactivates trainer	buttons on /admin/trainers	app/admin/trainers/TrainerActions.tsx	app/api/admin/trainers/[id]/status/route.ts	POST
Trainer creates a member	Save on /trainer/members/new	app/trainer/members/new/page.tsx	app/api/trainer/members/route.ts	POST 36 to 67
Save an assessment	Fitness Data tab, Save	app/trainer/members/[id]/FitnessDataTab.tsx, handleSubmit() 41	app/api/trainer/members/[id]/assessments/route.ts	POST
Run ML prediction	Predictions tab, Run	app/trainer/members/[id]/PredictionsTab.tsx, runPrediction() 20	app/api/assessments/[id]/predict/route.ts	POST 8 to 61

Feature	What the user clicks	Front end file (sends the request)	Back end file	Function, lines
ML model answers	(called by the route above)	none	FitWise_Prototype_Lab/ml/app.py	predict() 63 to 106
Train the ML model	run python train.py	none	FitWise_Prototype_Lab/ml/train.py	main() 136 to 262
Generate two plans	Plans tab, Generate	app/trainer/members/[id]/PlansTab.tsx, generate() 23	app/api/plans/generate/route.ts	POST 19 to 92
Rule based plan	(called by generate)	none	lib/ruleBased.ts	generateRuleBasedPlan() 103 to 136
Gemini plan	(called by generate)	none	lib/geminiGenerator.ts	generateGeminiPlan() 46 to 108
Check a plan	(called by generate and approve)	none	lib/validator.ts	validatePlan() 13 to 77
Approve a plan	Plans tab, Approve	PlansTab.tsx, approve() 83	app/api/plans/[id]/approve/route.ts	POST 10 to 92
Reject a plan	Plans tab, Reject	PlansTab.tsx, reject() 93	app/api/plans/[id]/reject/route.ts	POST
Member marks day done (web)	Complete Day on /member	app/member/CompleteDayButton.tsx, handleClick() 10	app/api/member/completions/route.ts	POST
Member marks day done (phone)	Finish on the workout screen	mobile/lib/screens/workout_complete_screen.dart, _save() 20	Supabase directly	insert into completions
Progress projection	Progress tab (web), Projected Progress (phone)	ProgressTab.tsx, mobile/lib/screens/projected_progress_screen.dart _load() 32	app/api/members/[id]/progress/route.ts + lib/progress.ts	GET 50 to 95, computeProgress() 36 to 62
Who can see which rows	everything	none	supabase/migrations/*.sql	create policy statements
Phone app start and login	opening the app	mobile/lib/main.dart, mobile/lib/screens/login_screen.dart	Supabase Auth	main() 14, AuthGate 73, _signIn() 20
Phone plan list	Plan tab	mobile/lib/screens/plan_tab.dart	Supabase directly	_load() 26 to 41
Phone workout timer	Start workout	mobile/lib/screens/active_workout_screen.dart	none (runs on phone)	_startRest() 39, _completeSet() 53, _nextExercise() 62

3. Folder structure

The repository holds the final app (FitWise_App) and the earlier experiments it grew from (FitWise_Prototype_Lab). Know this split well: reviewers often ask "why are there two of everything?"

```
FitWise HOUZ/
├── FitWise_App/                  <- the final system
|   ├── web/                      <- Next.js web app + API
|   |   ├── app/                  <- pages and routes (folder = URL)
|   |   |   ├── page.tsx          <- public landing page
|   |   |   ├── login/ signup/ pending/
|   |   |   ├── admin/            <- admin portal pages
|   |   |   ├── trainer/          <- trainer portal pages
|   |   |   ├── member/           <- member web portal
|   |   |   └── api/              <- server API routes (route.ts files)
|   |   ├── lib/                  <- shared logic, no UI
|   |   |   ├── ruleBased.ts      <- deterministic plan generator
|   |   |   ├── geminiGenerator.ts<- Gemini plan generator
|   |   |   ├── validator.ts      <- checks any plan against the rules
|   |   |   ├── progress.ts       <- 3 month weight scenario
|   |   |   ├── auth.ts authApi.ts<- role checks
|   |   |   ├── types.ts          <- shared TypeScript types
|   |   |   └── supabase/         <- server, browser and admin DB clients
|   |   ├── supabase/migrations/  <- 11 SQL files: tables + RLS policies
|   |   ├── scripts/              <- seed exercises, bootstrap admin/trainer
|   |   └── proxy.ts              <- refreshes the login cookie on every
request
|   └── mobile/                   <- Flutter member app
|       ├── lib/main.dart         <- app entry, Supabase init, routing
|       ├── lib/theme.dart        <- colours, fonts, shared widgets
|       └── lib/screens/          <- one file per screen
└── FitWise_Prototype_Lab/        <- the experiments done first
    ├── ml/                       <- Python ML (train.py, app.py, data,
artifacts)
    └── web/                      <- Rule vs Gemini comparison lab
```

The ML service is not copied into FitWise_App. The final web app calls the same FastAPI service from FitWise_Prototype_Lab/ml through the ML_API_URL setting, so there is one trained model and one place it lives.

One convention to explain if asked: in Next.js App Router the **folder path is the URL**. app/trainer/members/[id]/page.tsx is the page at /trainer/members/123, and app/api/plans/generate/route.ts answers POST /api/plans/generate. A folder in square brackets, such as [id], is a dynamic part of the URL.

4. Code walkthrough

This is the real code from the project, shortened only where lines repeat. Every comment marked with ← is an explanation added for you; open the same file and lines to see the original.

4A. Login and role checks

File: lib/auth.ts, lines 19 to 57. Used by every protected page.

```
export async function verifySession(): Promise<Session> {
  const supabase = await createClient();                    // ← DB client
acting as the logged in user (reads the login cookie)
  const { data: { user } } = await supabase.auth.getUser(); // ← ask
Supabase: who is this?

  if (!user) redirect("/login");                            // ← not logged
in: send to login page

  const { data: profile } = await supabase
    .from("profiles")                                        // ← look up this
user's row in profiles
    .select("role, status")
    .eq("id", user.id)
    .single();

  if (!profile) redirect("/login");
  return { userId: user.id, email: user.email, role: profile.role, status:
profile.status ?? "active" };
}

export function homeFor(session) {                           // ← decides
where each role belongs
  if (session.status !== "active") return "/pending";        // ← trainer not
approved yet
  if (session.role === "admin") return "/admin";
  if (session.role === "trainer") return "/trainer";
  return "/member";
}

export async function requireTrainer(): Promise<Session> {
  const session = await verifySession();
  if (session.role !== "trainer" || session.status !== "active")
```

```
    redirect(homeFor(session));                                  // ← wrong role:
bounce them to their own area
  return session;
}
```

File: lib/authApi.ts, lines 8 to 24. Same idea for API routes, but returns an error instead of redirecting.

```
async function requireRoleApi(role: "trainer" | "admin") {
  const supabase = await createClient();
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) {
    return { error: NextResponse.json({ error: "Unauthorized" }, { status:
401 }) };  // ← 401 = not logged in
  }
  const { data: profile } = await supabase.from("profiles").select("role,
status").eq("id", user.id).single();
  if (profile?.role !== role || profile?.status !== "active") {
    return { error: NextResponse.json({ error: "Forbidden" }, { status: 403
}) };     // ← 403 = logged in, wrong role
  }
  return { supabase, userId: user.id };                    // ← success: give
the route a DB client and the user id
}
```

4B. API route: run the ML prediction

File: app/api/assessments/[id]/predict/route.ts, lines 8 to 61. Called when the trainer clicks Run on the Predictions tab.

```
export async function POST(_request, { params }) {
  const auth = await requireTrainerApi();                  // ← only an
active trainer may continue
  if ("error" in auth) return auth.error;
  const { id } = await params;                              // ← the
assessment id from the URL

  const { data: assessment } = await auth.supabase
    .from("assessments").select("*").eq("id", id).single(); // ← RLS: returns
nothing if not this trainer's member
  if (!assessment) return NextResponse.json({ error: "Assessment not found"
}, { status: 404 });

  const mlUrl = process.env.ML_API_URL ?? "http://127.0.0.1:8000";
  const mlResponse = await fetch(`${mlUrl}/predict`, {     // ← call the
Python FastAPI service
```

```
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({                                    // ← the 8
features the model needs
      age: assessment.age, gender: assessment.gender,
      weight_kg: assessment.weight_kg, height_m: assessment.height_m,
      fat_percentage: assessment.fat_percentage, workout_frequency:
assessment.workout_frequency,
      experience_level: assessment.experience_level, bmi: assessment.bmi,
    }),
  });
  const mlBody = await mlResponse.json();

  const { data: prediction } = await auth.supabase
    .from("predictions")
    .insert({                                                 // ← save the
result so plans can use it later
      assessment_id: id,
      predicted_category: mlBody.predictedWorkoutType,      // ← e.g.
"Strength"
      confidence: mlBody.confidence,                        // ← highest
probability
      probabilities: mlBody.probabilities,                  // ← all 4
probabilities
      model_name: mlBody.modelName, model_version: mlBody.modelVersion,
    })
    .select().single();

  return NextResponse.json({ prediction });                 // ← back to the
Predictions tab
}
```

4C. API route: generate the two plans

File: app/api/plans/generate/route.ts, lines 19 to 92. Called when the trainer clicks Generate on the Plans tab. This is the centre of the whole project.

```
export async function POST(request: Request) {
  const auth = await requireTrainerApi();
  if ("error" in auth) return auth.error;
  const { assessmentId } = await request.json();

  // STEP 1: load the exercise catalogue (only active exercises)
  const { data: exerciseRows } = await
auth.supabase.from("exercises").select("*").eq("is_active", true);
  const exercises = exercisesFromDb(exerciseRows ?? []);   // ← converts DB
```

```
names (snake_case) to code names (camelCase)

  // STEP 2: load the assessment and its newest prediction
  const { data: assessment } = await
auth.supabase.from("assessments").select("*").eq("id",
assessmentId).single();
  const { data: prediction } = await auth.supabase
    .from("predictions").select("*").eq("assessment_id", assessmentId)
    .order("created_at", { ascending: false }).limit(1).single();  // ←
newest prediction only
  if (!prediction) return NextResponse.json({ error: "Run an ML prediction
for this assessment first." }, { status: 400 });

  // STEP 3: one profile object, given to BOTH generators (fair comparison)
  const profile = profileFromAssessment(assessment,
prediction.predicted_category);

  // STEP 4: run each generator; if one fails, the other still runs
  const results = [];
  try { results.push({ plan: generateRuleBasedPlan(profile, exercises) }); }
  catch (error) { results.push({ error: error.message }); }
  try { results.push({ plan: await generateGeminiPlan(profile, exercises) });
}
  catch (error) { results.push({ error: error.message }); }     // ← e.g.
Gemini quota or network error

  // STEP 5: validate each plan and save it as a draft
  const inserted = [];
  for (const result of results) {
    if (!result.plan) continue;
    const validation = validatePlan(result.plan, profile, exercises);  // ←
same checks for both
    const { data } = await auth.supabase.from("plans").insert({
      assessment_id: assessmentId,
      prediction_id: prediction.id,                          // ← links the
plan to the exact prediction
      generator: result.plan.generator,                      // ← "rule_based"
or "gemini"
      days: result.plan.days,                                // ← the plan
itself, stored as JSON
      validation,                                            // ← score and
violations, stored as JSON
      status: "draft",                                       // ← member
cannot see drafts
      /* plan_name, days_per_week, explanation, rules_applied also saved */
    }).select().single();
```

```
    inserted.push(data);
  }
  return NextResponse.json({ plans: inserted, errors: results.filter((r) =>
r.error).map((r) => r.error) });
}
```

4D. API route: approve a plan

File: app/api/plans/[id]/approve/route.ts, lines 10 to 92.

```
const { editedDays, overrideReason } = await request.json();   // ← both
optional
const { data: plan } = await auth.supabase.from("plans").select("*").eq("id",
id).single();
let finalDays = plan.days;
let finalValidation = plan.validation;

if (editedDays) {                                               // ← trainer
changed the plan
  const editedValidation = validatePlan(editedPlan, profile, exercises);  //
← check the edited version again
  if (!editedValidation.valid && !overrideReason)
    return NextResponse.json({ error: "Edited plan fails validation" }, {
status: 400 });
  await auth.supabase.from("plan_audit_log").insert({        // ← keep the
OLD version in the audit log
    plan_id: id, action: "edited", previous_days: plan.days, changed_by:
auth.userId });
  finalDays = editedDays;
  finalValidation = editedValidation;
}

if (!finalValidation?.valid && !overrideReason)               // ← invalid
plan needs a written reason
  return NextResponse.json({ error: "Plan fails validation" }, { status: 400
});

await auth.supabase.from("plans").update({
  days: finalDays, validation: finalValidation,
  status: "approved",                                         // ← from now
on the member can see it (RLS)
  approved_by: auth.userId, approved_at: new Date().toISOString(),
```

```
}).eq("id", id);

await auth.supabase.from("plan_audit_log").insert({ plan_id: id, action:
"approved", reason: overrideReason ?? null, changed_by: auth.userId });
```

4E. The database rule that protects it all

File: supabase/migrations/0004_exercises_plans.sql. This runs inside PostgreSQL, not in the app.

```
create policy "member reads own approved plans" on public.plans
  for select using (
    status = 'approved'                                    -- ← drafts and
rejected plans are invisible
    and exists (
      select 1 from public.assessments a
      where a.id = plans.assessment_id
        and a.member_id = auth.uid()                       -- ← and only
the logged in member's own plans
    )
  );
```

4F. Rule based generator

File: lib/ruleBased.ts. Read it top to bottom in this order.

Lines 8 to 13: the category order. The predicted category comes first, then fallbacks.

```
const categoryFallback = {
  Strength: ["Strength", "HIIT", "Yoga", "Cardio"],   // ← if ML says
Strength, Strength exercises go first
  Cardio:   ["Cardio", "HIIT", "Strength", "Yoga"],
  HIIT:     ["HIIT", "Cardio", "Strength", "Yoga"],
  Yoga:     ["Yoga", "Strength", "Cardio", "HIIT"],
};
```

Lines 15 to 38: the three safety filters and the sort (RULE-01 to RULE-04).

```
function equipmentAllowed(exercise, profile) {          // RULE-02
  return exercise.equipment.every((item) =>
profile.equipment.includes(item));
}                                                       // ← every item the
exercise needs must be in the member's list

function restrictionAllowed(exercise, profile) {        // RULE-03
  return !exercise.contraindicationTags.some((tag) =>
```

```
profile.restrictions.includes(tag));
}                                                          // ← e.g.
knee_high_load on the exercise + knee_high_load on the member = removed

function difficultyAllowed(exercise, profile) {           // RULE-04
  return exercise.difficulty.includes(profile.experienceLevel);
}                                                          // ← Burpees are
"advanced" only, so a beginner never gets them

function eligibleExercises(profile, exercises) {
  const categoryRank = categoryFallback[profile.predictedWorkoutType ??
"Strength"];
  return exercises
    .filter((e) => equipmentAllowed(e, profile) && restrictionAllowed(e,
profile) && difficultyAllowed(e, profile))
    .sort((a, b) => {                                      // RULE-01
      const categoryDiff = categoryRank.indexOf(a.category) -
categoryRank.indexOf(b.category);
      if (categoryDiff !== 0) return categoryDiff;         // ← sort by category
order first
      return a.id.localeCompare(b.id);                     // ← then by ID, so
the order is always the same
    });
}
```

Lines 40 to 62: sets, reps and rest.

```
function prescription(exercise, profile) {
  const levels = {
    beginner:     { sets: 2, reps: 10, rest: 75 },
    intermediate: { sets: 3, reps: 10, rest: 60 },
    advanced:     { sets: 4, reps: 8,  rest: 60 },
  };
  const p = levels[profile.experienceLevel];
  if (exercise.category === "Cardio" || exercise.category === "HIIT" ||
exercise.category === "Yoga") {
    return { exerciseId: exercise.id, durationMinutes:
exercise.estimatedMinutes,
             restSeconds: exercise.category === "HIIT" ? 45 : 30 };   // ←
timed, not sets and reps
  }
  return { exerciseId: exercise.id, sets: p.sets, reps: p.reps, restSeconds:
p.rest };  // ← Strength
}
```

Lines 64 to 97: build one day (RULE-05).

```
function buildDay(day, profile, pool) {
  const selected = [];
  const usedPrimaryMuscles = new Set();
  let estimated = 8;                                    // ← 8 minutes
reserved for warm up

  for (const exercise of pool) {
    if (estimated + exercise.estimatedMinutes >
profile.sessionDurationMinutes - 5) continue;  // ← would run over time: skip
    const primary = exercise.muscles[0];
    if (usedPrimaryMuscles.has(primary) && selected.length < 4) continue;  //
← avoid hitting the same muscle twice early on
    selected.push(exercise);
    usedPrimaryMuscles.add(primary);
    estimated += exercise.estimatedMinutes;
    if (selected.length >= 6) break;                    // ← maximum 6
exercises a day
  }
  // lines 80 to 88: if fewer than 3 were picked, a second loop tops the day
up to 3
  return { day, title: `${profile.predictedWorkoutType} Session ${day}`,
focus: profile.goal.replaceAll("_", " "),
           estimatedDurationMinutes: Math.min(profile.sessionDurationMinutes,
estimated + 5),
           exercises: selected.map((e) => prescription(e, profile)) };
}
```

Lines 103 to 136: the main function (RULE-06).

```
export function generateRuleBasedPlan(profile, exercises = EXERCISES) {
  if (!profile.predictedWorkoutType) throw new Error("ML predictedWorkoutType
is required before plan generation.");
  const pool = eligibleExercises(profile, exercises);    // ← filtered and
sorted list
  if (pool.length < 3) throw new Error("Not enough approved exercises satisfy
this profile's equipment/restrictions.");

  const days = Array.from({ length: profile.daysPerWeek }, (_, i) => {
    const offset = (i * 3) % pool.length;                // ← day 1 starts at
0, day 2 at 3, day 3 at 6 ...
    const rotated = [...pool.slice(offset), ...pool.slice(0, offset)];  // ←
rotate the list so days differ
    return buildDay(i + 1, profile, rotated);
  });
  return { generator: "rule_based", planName: `Deterministic
${profile.predictedWorkoutType} Plan`,
           days, rulesApplied: ["RULE-01: ...", "RULE-06: ..."] /* plus
predictedWorkoutType, daysPerWeek, explanation */ };
}
```

4G. Gemini generator

File: lib/geminiGenerator.ts.

Lines 5 to 44: the JSON schema. Tells Gemini the exact shape of the answer.

```
const workoutPlanSchema = {
  type: "object",
  properties: {
    predictedWorkoutType: { type: "string", enum: ["Cardio", "Strength",
"HIIT", "Yoga"] },  // ← only these 4 allowed
    daysPerWeek: { type: "integer", minimum: 1, maximum: 7 },
    days: { type: "array", items: { /* day, title, focus,
estimatedDurationMinutes, exercises[] */ } },
  },
  required: ["planName", "predictedWorkoutType", "daysPerWeek",
"explanation", "days"],
};
```

Lines 46 to 108: build the prompt, call Gemini, clean the answer.

```
export async function generateGeminiPlan(profile, exercises = EXERCISES) {
  const apiKey = process.env.GEMINI_API_KEY;             // ← secret key,
only on the server
  const model = process.env.GEMINI_MODEL ?? "gemini-3.6-flash";
```

```
  const ai = new GoogleGenAI({ apiKey });

  const prompt = `
  CRITICAL CONSTRAINTS:
  1. Use ONLY exercise IDs from APPROVED_EXERCISES. Never invent an exercise
ID or name.
  2. Preserve the ML predicted workout category as predictedWorkoutType.
  3. Produce exactly the requested number of workout days.
  ... (9 rules in total)
  PROFILE: ${JSON.stringify(profile, null, 2)}                ← the member's
data
  APPROVED_EXERCISES: ${JSON.stringify(approvedExercises, null, 2)}  ← the
only exercises allowed
  `;

  const interaction = await ai.interactions.create({
    model,
    input: prompt,
    response_format: { type: "text", mime_type: "application/json", schema:
workoutPlanSchema },  // ← force JSON in our shape
  });

  const parsed = JSON.parse(interaction.output_text);    // ← text →
JavaScript object
  return {
    ...parsed,
    generator: "gemini",
    predictedWorkoutType: profile.predictedWorkoutType,  // ← overwrite:
Gemini can never change the ML category
  };
}
```

4H. Validator

File: lib/validator.ts, lines 13 to 77. Same function checks both plans.

```
export function validatePlan(plan, profile, exercises = EXERCISES) {
  const exerciseMap = new Map(exercises.map((e) => [e.id, e]));  // ← quick
lookup: ID → exercise
  const violations = [];
  let passed = 0, total = 0;

  const check = (ok, message) => {                       // ← helper used for
every rule
    total += 1;
    if (ok) passed += 1;
```

```
    else violations.push(message);                          // ← remember what
failed, in plain English
  };

  check(plan.predictedWorkoutType === profile.predictedWorkoutType, "Plan
does not preserve the ML-predicted workout category.");
  check(plan.days.length === profile.daysPerWeek, `Expected
${profile.daysPerWeek} workout days ...`);

  plan.days.forEach((day) => {
    check(day.estimatedDurationMinutes <= profile.sessionDurationMinutes,
`Day ${day.day} exceeds requested session duration.`);
    check(day.exercises.length >= 2, `Day ${day.day} contains too few
exercises.`);
    const ids = day.exercises.map((e) => e.exerciseId);
    check(new Set(ids).size === ids.length, `Day ${day.day} contains
duplicate exercise IDs.`);  // ← a Set removes duplicates

    day.exercises.forEach((item) => {
      const exercise = exerciseMap.get(item.exerciseId);
      check(Boolean(exercise), `Unknown exercise ID returned:
${item.exerciseId}`);  // ← catches invented exercises
      if (!exercise) return;
      check(exercise.difficulty.includes(profile.experienceLevel), ...);
// ← same 3 rules as the generator
      check(exercise.equipment.every((e) => profile.equipment.includes(e)),
...);
      check(!exercise.contraindicationTags.some((tag) =>
profile.restrictions.includes(tag)), ...);
      if (item.sets !== undefined) check(Number.isInteger(item.sets) &&
item.sets >= 1 && item.sets <= 10, ...);
      if (item.reps !== undefined) check(Number.isInteger(item.reps) &&
item.reps >= 1 && item.reps <= 50, ...);
      check(item.restSeconds >= 0 && item.restSeconds <= 300, ...);
    });
  });

  const score = total === 0 ? 0 : Math.round((passed / total) * 10000) / 100;
// ← e.g. 47 of 50 checks = 94.00
  return { valid: violations.length === 0, score, checksPassed: passed,
totalChecks: total, violations };
}
```

4I. Progress scenario

File: lib/progress.ts, lines 13 to 62.

```
const BASE_MONTHLY_CHANGE_KG = {
  weight_loss: -2, muscle_gain: 0.25,                    // ← fixed assumed
rates, kg per month
  endurance: 0, general_fitness: 0, flexibility: 0,      // ← not weight
goals, so flat
};

export function computeProgress(baselineWeightKg, goal, adherenceFactor) {
  const clampedAdherence = Math.max(0, Math.min(1, adherenceFactor));  // ←
keep between 0 and 1
  const monthlyChangeRateKg = BASE_MONTHLY_CHANGE_KG[goal];
  const projectedChange = (months) => monthlyChangeRateKg * clampedAdherence
* months;
  // ← example: 80 kg, weight_loss, 50% adherence → month 3 = 80 + (-2 × 0.5
× 3) = 77 kg
  const projections = {
    month1: round1(baselineWeightKg + projectedChange(1)),
    month2: round1(baselineWeightKg + projectedChange(2)),
    month3: round1(baselineWeightKg + projectedChange(3)),
  };
  return { baselineWeightKg, goal, adherenceFactor: clampedAdherence,
projections, targetWeightKg: projections.month3 /* + assumptions, disclaimer
*/ };
}
```

The adherence number comes from app/api/members/[id]/progress/route.ts, line 91: completedDays / totalDays (rows in completions divided by the days in all approved plans).

4J. ML training

File: FitWise_Prototype_Lab/ml/train.py. Run once with python train.py; it produces the model file the API loads.

Lines 26 to 42: what goes in and what comes out.

```
TARGET = "Workout_Type"                          # ← what we predict: Cardio,
HIIT, Strength or Yoga
FEATURES = [                                     # ← the 8 inputs, all known at
a first assessment
    "Age", "Gender", "Weight (kg)", "Height (m)", "Fat_Percentage",
    "Workout_Frequency (days/week)", "Experience_Level", "BMI",
]
CATEGORICAL = ["Gender"]                         # ← text column, needs one hot
encoding
NUMERIC = [c for c in FEATURES if c not in CATEGORICAL]  # ← the other 7
RANDOM_STATE = 42                                # ← fixed seed so results can
be repeated
```

Lines 45 to 62: load_data(). Reads the CSV, checks the columns exist, removes duplicate rows.

Lines 65 to 85: build_preprocessor().

```
numeric_pipe = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),   # ← fill any missing
number with the median
    ("scaler", StandardScaler()),                    # ← rescale so every
feature has mean 0, std 1
])
categorical_pipe = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),  # ← Gender → two 0/1
columns
])
return ColumnTransformer([("num", numeric_pipe, NUMERIC), ("cat",
categorical_pipe, CATEGORICAL)])
```

Lines 88 to 133: candidate_models(). Five models, each with a small list of settings to try.

```
"Logistic Regression": (
    LogisticRegression(max_iter=2000, class_weight="balanced",
random_state=RANDOM_STATE),
    {"model__C": [0.01, 0.1, 1, 10]},            # ← GridSearchCV tries all
4 values of C
),
"Decision Tree": (..., {"model__max_depth": [3, 5, 7, None],
"model__min_samples_leaf": [1, 2, 5]}),
"Random Forest": (...), "Support Vector Machine": (...), "Gradient Boosting":
(...),
```

Lines 136 to 262: main(), the training run.

```
X = df[FEATURES]; y = df[TARGET]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=RANDOM_STATE)   # ← 80%
train, 20% test, same class mix in both

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)  #
← 5 fold cross validation

baseline = DummyClassifier(strategy="most_frequent")  # ← always guesses the
biggest class: the bar to beat

for name, (estimator, param_grid) in candidate_models().items():
    pipeline = Pipeline(steps=[("preprocess", build_preprocessor()),
("model", estimator)])  # ← preprocessing + model together
    search = GridSearchCV(pipeline, param_grid=param_grid,
scoring="f1_macro", cv=cv, refit=True)
    search.fit(X_train, y_train)              # ← tries every setting, 5
folds each, training data only
    pred = search.best_estimator_.predict(X_test)  # ← test set used once,
for reporting

best_row = max(tuned_rows, key=lambda r: r["cv_macro_f1_mean"])   # line 216
← winner picked on CV score, not test score

joblib.dump(best_pipeline, ARTIFACT_DIR / "workout_pipeline.joblib")  # line
255 ← save the trained model to a file
# also writes model_metadata.json (name, version, features) and
model_results.json (all scores)
```

4K. ML API

File: FitWise_Prototype_Lab/ml/app.py. Started with uvicorn app:app --port 8000.

Lines 21 to 29: input checking. Pydantic rejects bad values with a 422 error before the model runs.

```
class PredictionRequest(BaseModel):
    age: int = Field(ge=16, le=100)            # ← ge = at least, le = at
most
    gender: Literal["Male", "Female"]          # ← only these two strings
accepted
    weight_kg: float = Field(gt=25, le=300)
    height_m: float = Field(gt=1.2, le=2.3)
    fat_percentage: float = Field(ge=3, le=60)
    workout_frequency: int = Field(ge=1, le=7)
    experience_level: int = Field(ge=1, le=3)
    bmi: float = Field(gt=10, le=70)
```

Lines 32 to 42: load the model once when the server starts (lifespan), not on every request, so predictions are fast.

Lines 63 to 106: the /predict endpoint.

```
@app.post("/predict")
def predict(payload: PredictionRequest):
    frame = pd.DataFrame([{                    # ← one row with the SAME
column names used in training
        "Age": payload.age, "Gender": payload.gender, "Weight (kg)":
payload.weight_kg, ...
    }])
    prediction = str(model.predict(frame)[0])    # ← e.g. "Strength" (the
pipeline scales and encodes first)
    probs = model.predict_proba(frame)[0]        # ← probability of each of
the 4 classes
    probabilities = {c: round(float(p), 4) for c, p in zip(model.classes_,
probs)}
    confidence = round(max(probabilities.values()), 4)   # ← confidence = the
highest probability
    return {
        "predictedWorkoutType": prediction, "confidence": confidence,
"probabilities": probabilities,
        "modelName": metadata.get("model_name"), "modelVersion":
metadata.get("model_version"), ...
    }
```

4L. Flutter app: start up and login

File: FitWise_App/mobile/lib/main.dart.

Lines 14 to 22: main(), the first code that runs.

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();     // ← required before any
async work at start up
  await dotenv.load();                           // ← read the .env file
(Supabase URL and key)
  await Supabase.initialize(
    url: dotenv.env['SUPABASE_URL']!,
    publishableKey: dotenv.env['SUPABASE_ANON_KEY']!,  // ← public key; RLS
still protects the data
  );
  runApp(const FitWiseApp());                    // ← start drawing the app
}
final supabase = Supabase.instance.client;       // ← one shared client used
by every screen
```

Lines 73 to 86: AuthGate, which decides login screen or home screen.

```
class AuthGate extends StatelessWidget {
  Widget build(BuildContext context) {
    return StreamBuilder<AuthState>(
      stream: supabase.auth.onAuthStateChange,   // ← fires every time the
user logs in or out
      builder: (context, snapshot) {
        final session = supabase.auth.currentSession;
        return session == null ? const LoginScreen() : const HomeShell();  //
← no session: login; session: app
      },
    );
  }
}
```

File: lib/screens/login_screen.dart, lines 20 to 37: _signIn().

```
await supabase.auth.signInWithPassword(
  email: _emailController.text.trim(),           // ← text typed in the Email
box
  password: _passwordController.text,
);
// no Navigator call needed: AuthGate above sees the new session and shows
HomeShell
```

4M. Flutter app: loading the plan

File: lib/screens/plan_tab.dart, lines 26 to 41: _load().

```
Future<void> _load() async {
  setState(() => _loading = true);              // ← show a spinner
  try {
    final plans = await
supabase.from('plans').select('*').order('created_at', ascending: false);
    // ↑ no "status = approved" filter here: the database RLS policy only
returns approved plans
    final completions = await supabase.from('completions').select('plan_id,
day_number');
    setState(() {
      _plans = List<Map<String, dynamic>>.from(plans);
      _completedKeys = {for (final c in completions)
'${c['plan_id']}:${c['day_number']}'};  // ← e.g. "abc:2" = day 2 done
    });
  } catch (e) {
    setState(() => _error = e.toString());       // ← show the error on
screen
  } finally {
    if (mounted) setState(() => _loading = false);  // ← hide the spinner
  }
}
```

4N. Flutter app: the workout timer

File: lib/screens/active_workout_screen.dart, lines 39 to 75.

```
void _startRest() {
  final rest = (_exercises[_exerciseIndex]['restSeconds'] as num?)?.toInt()
?? 60;  // ← rest time from the plan
  setState(() { _resting = true; _restSeconds = rest; });
  _timer = Timer.periodic(const Duration(seconds: 1), (t) {   // ← runs once
every second
    if (_restSeconds <= 1) { t.cancel(); setState(() => _resting = false); }
// ← rest over
    else { setState(() => _restSeconds -= 1); }               // ← count
down and redraw
  });
}

void _completeSet() {                            // ← member taps "Complete
set"
  if (_setIndex < _totalSets) {
    setState(() => _setIndex += 1);
    _startRest();                                // ← more sets left: rest,
then next set
```

```
  } else {
    _nextExercise();                                  // ← last set done: move on
  }
}

void _nextExercise() {
  _timer?.cancel();
  if (_exerciseIndex < _exercises.length - 1) {
    setState(() { _exerciseIndex += 1; _setIndex = 1; _resting = false; });
  } else {
    _finish();                                        // ← last exercise: go to
WorkoutCompleteScreen
  }
}
```

4O. Flutter app: saving a finished workout

File: lib/screens/workout_complete_screen.dart, lines 20 to 37: _save().

```
final userId = supabase.auth.currentUser!.id;
await supabase.from('completions').insert({     // ← one row = one day
completed
  'plan_id': widget.plan['id'],
  'member_id': userId,                            // ← RLS rejects the insert
if this is not the logged in user
  'day_number': widget.day['day'],
});
Navigator.of(context).pushAndRemoveUntil(         // ← back to the home
screen, clearing the workout screens
  MaterialPageRoute(builder: (_) => const HomeShell()), (route) => false);
```

4P. Flutter app: calling the web API for progress

File: lib/screens/projected_progress_screen.dart, lines 32 to 59: _load().

```
final session = supabase.auth.currentSession!;
final baseUrl = kIsWeb ? (dotenv.env['WEB_API_URL_WEB'] ??
'http://localhost:3002') : dotenv.env['WEB_API_URL'];
// ↑ Android emulator reaches the PC through 10.0.2.2; a browser build uses
localhost
final response = await http.get(
  Uri.parse('$baseUrl/api/members/$userId/progress'),
  headers: {'Authorization': 'Bearer ${session.accessToken}'},  // ← phone
has no cookie, so it sends its login token
);
final body = jsonDecode(response.body);          // ← { projections: {
month1, month2, month3 }, adherenceFactor, ... }
```

On the server, app/api/members/[id]/progress/route.ts lines 29 to 48 (resolveClient) reads that Bearer token and builds a Supabase client from it, so RLS treats the request as that member.

5. Backend explained

The backend is the Next.js API routes plus Supabase plus the Python ML service. Security is enforced twice: once in the route code (role checks) and again inside the database (Row Level Security), so a mistake in one layer is caught by the other.

5.1 Database tables

All tables are created by the 11 numbered SQL files in supabase/migrations. Each migration adds to the one before it, which is why columns like status and email on profiles only appear in 0011.

Table	One row is	Key columns	Created in
profiles	A user account (any role)	id (same as auth.users id), role, status, full_name, email	0001, extended 0011
members	A member and their trainer	profile_id, trainer_id, phone, date_of_birth	0002, 0008
assessments	One fitness assessment of a member	age, gender, weight_kg, height_m, fat_percentage, bmi, goal, days_per_week, equipment[], restrictions[]	0002, 0008

Table	One row is	Key columns	Created in
predictions	One ML result for an assessment	predicted_category, confidence, probabilities, model_version	0003
exercises	One approved exercise in the catalogue	id (e.g. STR-001), category, difficulty[], equipment[], contraindication_tags[], is_active	0004
plans	One generated weekly plan	generator (rule_based or gemini), days (JSON), validation (JSON), status (draft, approved, rejected)	0004, 0005
plan_audit_log	One trainer action on a plan	action (edited, approved, rejected), reason, previous_days	0005
completions	A member finishing one plan day	plan_id, member_id, day_number, notes	0006
admin_audit_log	One admin action	action, target_id, details	0011

The chain to remember: **member → assessment → prediction → plans → completions**. Every plan points back to the assessment and prediction it came from, so any plan can be traced to the exact data and model version behind it.

A database trigger, handle_new_user (0001, updated in 0011), automatically inserts a profiles row with role member every time Supabase Auth creates a user. Code that creates a trainer then updates that row to role trainer.

5.2 Authentication and roles

Login is handled by Supabase Auth (email and password). On the web, the session lives in a cookie; on mobile, the Flutter SDK stores the token and sends it as an Authorization: Bearer header.

- **proxy.ts** runs before every request and refreshes the Supabase cookie so it never expires mid session. (Next.js 16 renamed middleware.ts to proxy.ts.)
- **lib/auth.ts** is used by pages. verifySession() reads the user and their profile; requireTrainer() and requireAdmin() redirect anyone with the wrong role. homeFor() sends each role to its own area (/admin, /trainer, /member, or /pending if not yet approved).
- **lib/authApi.ts** is the same check for API routes, but it returns a 401 (not logged in) or 403 (wrong role) JSON response instead of redirecting.

- **lib/supabase/** has three clients. server.ts and client.ts use the publishable key and act as the logged in user, so RLS applies. admin.ts uses the secret service role key, which bypasses RLS; it is only used for things RLS cannot express, such as creating a new login for a member, and it is marked server-only so it can never reach the browser.

Row Level Security (RLS) means PostgreSQL itself filters rows by who is asking. For example, the policy "trainer manages own members assessments" only lets a trainer see an assessment if that member's trainer_id equals the trainer's own id (auth.uid()). So if a trainer requests another trainer's assessment ID, the database simply returns nothing and the route answers 404.

Migration 0011 adds two more safeguards: current_app_role() returns a role only for active accounts (so a pending trainer gets no trainer rights), and the trigger protect_profile_privileges stops any user from changing their own role, status or email; only the server can.

5.3 API routes

Method and URL	Who can call it	What it does
POST /api/auth/trainer-signup	Anyone	Creates a trainer account with status pending
GET, POST /api/trainer/members	Trainer	Lists own members; creates a member login with a one time temporary password
GET, POST /api/trainer/members/[id]/assessments	Trainer	Lists or saves a member's assessment
POST /api/assessments/[id]/predict	Trainer	Sends the assessment to the ML service and saves the prediction
POST /api/plans/generate	Trainer	Runs both generators, validates both plans, saves them as drafts
POST /api/plans/[id]/approve	Trainer	Approves a plan (optionally edited); invalid plans need an override reason
POST /api/plans/[id]/reject	Trainer	Rejects a plan; a reason is required

Method and URL	Who can call it	What it does
POST /api/member/completions	Member	Records that a plan day was completed
GET /api/members/[id]/progress	Member (self) or trainer	Adherence and 3 month weight scenario; accepts cookie or Bearer token (used by mobile)
GET /api/exercises, PATCH /api/exercises/[id]	Admin edits	Reads or edits catalogue fields (difficulty, equipment, tags, active)
POST /api/admin/trainers, /api/admin/trainers/[id]/status	Admin	Adds a trainer; approve, reject, deactivate or reactivate one
POST /api/admin/members/[id]/reassign	Admin	Moves a member to another active trainer
GET /api/health	Anyone	Checks Supabase and the ML service are reachable

Every protected route starts the same way, which is worth quoting in the viva:

```
const auth = await requireTrainerApi();
if ("error" in auth) return auth.error;   // 401 or 403
// from here on, auth.supabase acts as this trainer, with RLS applied
```

5.4 The plan pipeline (the core of the project)

ML prediction (app/api/assessments/[id]/predict/route.ts). Loads the assessment, sends the 8 features to the ML service's /predict endpoint, and saves the returned category, confidence and per class probabilities in predictions, together with the model name and version.

Plan generation (app/api/plans/generate/route.ts). Loads the active exercises from the database, the assessment and its latest prediction, converts them into one MemberProfile object (lib/profileFromAssessment.ts), then gives that same profile and catalogue to both generators. Each generator runs inside its own try and catch, so if Gemini fails the rule based plan is still saved. Each plan is validated and stored with status draft.

Rule based generator (lib/ruleBased.ts). Fully deterministic: the same input always gives the same plan. It applies six named rules:

1. RULE-01: put exercises from the ML predicted category first (a fallback order fills the rest, e.g. Strength, HIIT, Yoga, Cardio).
2. RULE-02: remove exercises needing equipment the member does not have.
3. RULE-03: remove exercises whose contraindication tag (e.g. knee_high_load) matches a member restriction.
4. RULE-04: keep only exercises approved for the member's experience level.
5. RULE-05: keep each day within the session length (8 minutes are reserved for warm up).
6. RULE-06: rotate the exercise list by 3 places each day, so days differ but stay reproducible.

Sets, reps and rest come from the experience level: beginner 2 x 10 with 75 s rest, intermediate 3 x 10 with 60 s, advanced 4 x 8 with 60 s. Cardio, HIIT and Yoga use minutes instead of sets.

Gemini generator (lib/geminiGenerator.ts). Sends Gemini the profile, the approved exercise list and 9 numbered constraints (use only listed IDs, keep the predicted category, exact number of days, and so on). It also passes a JSON schema as response_format, so Gemini must answer in the exact WorkoutPlan shape. After parsing, the code overwrites predictedWorkoutType with the ML value, so Gemini cannot change the category.

Validator (lib/validator.ts). Checks any plan, from either generator, against the member's profile. Checks include: category preserved, correct number of days, each day within the time limit, at least 2 exercises per day, no duplicate exercises, every exercise ID exists, difficulty, equipment and restrictions respected, and sensible numbers (sets 1 to 10, reps 1 to 50, rest 0 to 300 s). It returns valid, a score (checks passed as a percentage) and a list of violations. The number bounds were added after Gemini once returned a sets value of about 4 quintillion.

Approval (app/api/plans/[id]/approve/route.ts). The trainer can approve as is or edit the days first. Edited days are validated again. An invalid plan can only be approved with an overrideReason, and every edit, approval and rejection is written to plan_audit_log with the previous version. Only approved plans are visible to the member, enforced by the RLS policy "member reads own approved plans".

Progress scenario (lib/progress.ts). Not a prediction model. It takes a fixed monthly rate (weight_loss −2 kg, muscle_gain +0.25 kg, other goals 0), multiplies it by the member's adherence (completed days divided by planned days, capped 0 to 1) and projects months 1, 2 and 3. It is labelled a scenario because the dataset has no repeated measurements of the same person over time.

5.5 The ML service

FitWise_Prototype_Lab/ml/train.py trains the model; app.py serves it.

- **Data**: Kaggle "Gym Members Exercise Tracking" CSV, 973 rows after removing duplicates; 778 for training, 195 for testing (80/20, stratified, random_state 42).
- **Features (8)**: Age, Gender, Weight, Height, Fat percentage, Workout frequency, Experience level, BMI. Only details a brand new member can give are used ("new_member_safe_v1"), so nothing that needs past workout logs.
- **Target**: Workout_Type, one of Cardio, HIIT, Strength, Yoga.
- **Preprocessing** (inside a scikit learn Pipeline): numbers get median imputation and StandardScaler; Gender gets one hot encoding. Keeping preprocessing in the pipeline means the exact same steps run at prediction time.
- **Model selection**: five models (Logistic Regression, Decision Tree, Random Forest, SVM, Gradient Boosting) each tuned with GridSearchCV and 5 fold stratified cross validation, scored on macro F1, plus a majority class baseline. The winner is chosen on cross validation score, not the test set, to avoid tuning to the test data.
- **Saved outputs**: workout_pipeline.joblib (the trained pipeline), model_metadata.json and model_results.json.
- **app.py**: FastAPI loads the pipeline once at start up. POST /predict validates input ranges with Pydantic (e.g. age 16 to 100, BMI over 10 and up to 70), then returns the category, confidence (highest probability), all four probabilities, model name and version.

Model	Best settings	CV macro F1	Test accuracy	Test macro F1
Logistic Regression (selected)	C = 0.1	0.284	0.236	0.222
Support Vector Machine	C = 10, gamma auto	0.278	0.221	0.222
Decision Tree	no depth limit	0.258	0.195	0.193
Gradient Boosting	100 trees, depth 2, rate 0.05	0.258	0.190	0.164
Random Forest	200 trees, depth 10	0.258	0.215	0.216
Baseline (majority class)	none	none	0.267	0.105

Be ready for this: the scores are low, close to random for four classes (0.25). Section 8 gives a prepared answer.

6. Front ends: web portals and mobile app

The web app serves admins and trainers (members can also use it), and the Flutter app is the member's main tool. Both talk to the same Supabase database, so a plan a trainer approves on the web appears on the phone straight away.

6.1 Web portals (Next.js)

Each portal is a folder under app/ with a layout.tsx that calls requireTrainer(), requireAdmin() or, for members, verifySession() first, so every page inside is protected in one place.

Portal	Main pages	Notes
Public	/ (landing), /login, /signup, /pending	Signup creates a pending trainer; /pending is shown until an admin approves
Admin	/admin, trainers, trainers/new, members, exercises, activity	Approve or deactivate trainers, reassign members, edit the exercise catalogue, view the audit log
Trainer	/trainer (dashboard), members, members/new, members/[id], plans, predictions, analytics, alerts, settings	The member detail page has six tabs: Overview, Fitness Data, Predictions, Plans, Progress, Notes
Member	/member	Shows the approved plan with a Complete Day button

Two kinds of React components are used, and reviewers often ask the difference:

- **Server Components** (the default, e.g. page.tsx) run on the server, read the database directly and send finished HTML. No database keys reach the browser.
- **Client Components** (files starting with "use client", e.g. PlansTab.tsx, CompleteDayButton.tsx) run in the browser for buttons and forms. They never touch the database directly; they call the API routes with fetch, for example fetch("/api/plans/generate").

The trainer's workflow on the member page follows the pipeline in order: Fitness Data tab saves an assessment, Predictions tab runs the ML model, Plans tab generates both plans side by side with their validation scores and lets the trainer approve or reject.

6.2 Mobile app (Flutter)

The Flutter app is for members only. It reads data straight from Supabase using the member's own login, so RLS decides what they see.

Start up (lib/main.dart):

1. Load the .env file (Supabase URL and key, web API URL) with flutter_dotenv.
2. Supabase.initialize(...) connects to the same Supabase project as the web app.
3. SplashGate shows the splash screen for 1.4 s and checks SharedPreferences to see if onboarding was already seen.
4. AuthGate listens to supabase.auth.onAuthStateChange. No session shows LoginScreen; a session shows HomeShell. Because it is a stream, logging in or out switches screens automatically with no manual navigation.

Screens (lib/screens/):

Screen	Purpose	Data source
HomeShell	Bottom navigation with 5 tabs, kept alive with an IndexedStack	none
HomeTab	Greeting, today's session, latest recommendation, trainer note	profiles, plans, completions, predictions, assessments
PlanTab	All plan days, tap one for SessionDetailsScreen	plans, completions
WorkoutsTab	History of completed workouts	completions
ProgressTab	Weight history, completed count, link to projection	assessments, plans, completions
ProfileTab	Personal details, links to settings, achievements, messages; sign out	profiles, assessments
ActiveWorkoutScreen	Guides set by set with a rest countdown (Timer.periodic)	the selected plan day
WorkoutCompleteScreen	Rating, then saves the completion	inserts into completions

Screen	Purpose	Data source
RecommendationScreen	ML category, confidence and all probabilities	predictions
ProjectedProgressScreen	3 month weight scenario	GET /api/members/[id]/progress
Splash, Onboarding, Login, Settings, Notifications, Messages, Achievements	Supporting screens	various

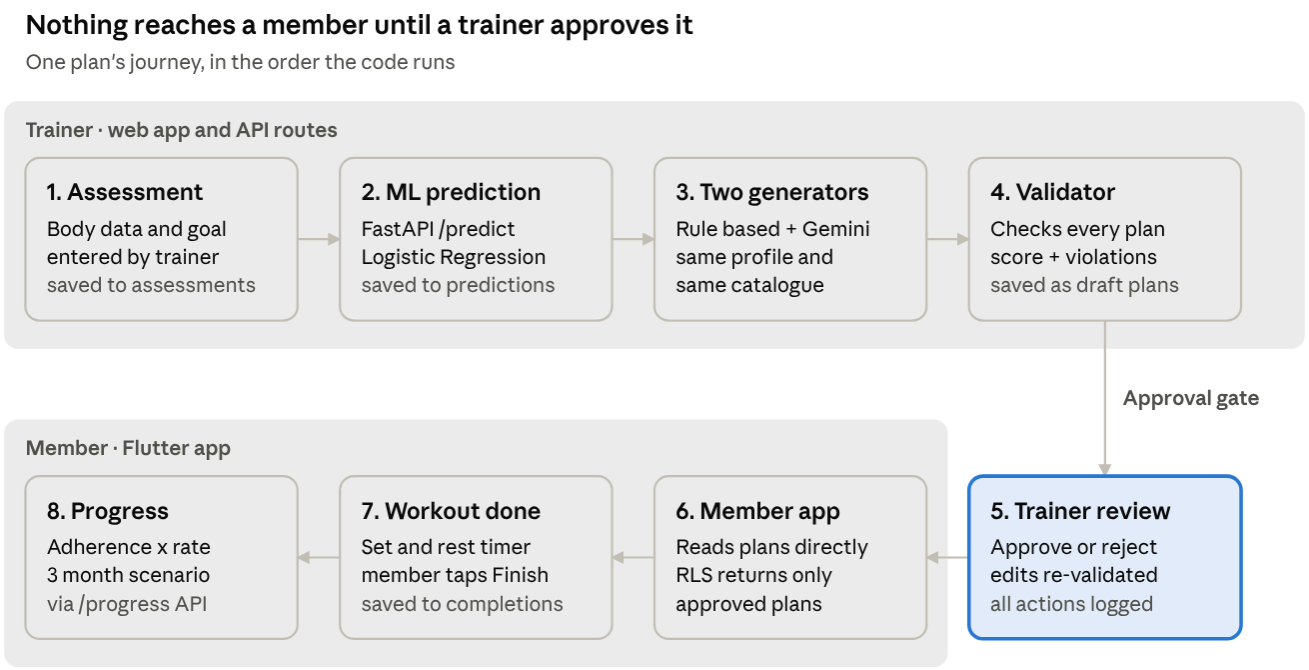
Two details worth explaining:

- **The mobile app never filters plans by status.** PlanTab runs supabase.from('plans').select('*'), yet a member only ever receives approved plans, because the RLS policy "member reads own approved plans" filters them in the database. This is a good example of security living in the database rather than in app code.
- **Only the progress screen calls the Next.js API**, because the scenario logic lives in lib/progress.ts on the server. It sends the Supabase access token as Authorization: Bearer, since a phone has no browser cookie. The route builds a Supabase client from that token so RLS still applies, and it allows CORS so the Flutter web build can call it too.

State is handled with plain StatefulWidget and setState: each screen loads its data in initState, keeps a _loading flag and an _error message, and rebuilds when data arrives. No state library (Provider, Bloc) is used, which keeps each screen self contained and easy to follow. lib/theme.dart holds the shared colours (FitWiseColors), fonts (Google Fonts) and the FitWiseBrand widget so all screens look consistent.

7. Data flow: one plan from start to finish

A plan passes through eight steps, and the trainer's approval in step 5 is the gate between the AI side and the member. If a reviewer asks you to "walk through the system", follow this picture.



end to end data flow · 8 steps, 1 approval gate

Steps 1 to 5 run in the web app's API routes as the trainer clicks through the member page tabs; steps 6 to 8 happen in the Flutter app. Each step saves its output to its own table, so every approved plan can be traced back to the exact assessment, prediction and model version behind it. A rejected plan stops at step 5 and never reaches the member.

8. Likely reviewer questions

Practise saying these answers out loud in your own words; each one points to the file to open if the reviewer asks you to show the code.

Architecture and design

Q: Why did you use both an ML model and a generator? Why not let one do everything? The ML model only answers one narrow question: which category suits this person. Building a full plan needs hard rules (equipment, injuries, time), which a classifier cannot guarantee. So the model picks the direction and the generators build the plan inside strict limits. Show: app/api/plans/generate/route.ts.

Q: Why two generators, rule based and Gemini? To compare a deterministic approach with a generative AI approach under fair conditions. Both receive the same profile, the same prediction and the same exercise catalogue, and both are checked by the same validator, so differences come from the generator alone. The trainer sees both and chooses.

Q: How do you stop Gemini from inventing exercises or ignoring injuries? Three layers. First, the prompt lists only approved exercises and nine explicit constraints. Second, a JSON schema forces the answer into the WorkoutPlan shape. Third, and most important, the validator checks every exercise ID, restriction, equipment item and number afterwards, and the trainer must approve. Gemini output is never trusted on its own. Show: lib/geminiGenerator.ts and lib/validator.ts.

Q: Why Next.js and Supabase? Next.js lets the pages and the API live in one TypeScript project, and Server Components keep secrets on the server. Supabase gives PostgreSQL, authentication and Row Level Security in one service, and has official SDKs for both Next.js and Flutter, so the web and mobile apps share one database and one login system.

Q: Why is the ML part in Python and separate from the web app? scikit learn is the standard tool for this kind of model and runs in Python. Wrapping it in FastAPI turns it into a small independent service the web app calls over HTTP (ML_API_URL), so the model can be retrained or replaced without touching the web code.

Machine learning

Q: Your model's accuracy is about 24%. Isn't that bad? Yes, and the project reports it honestly instead of hiding it. With four classes, random guessing gives about 25%. The dataset records which workout people happened to do, not which workout was best for them, so body measurements carry very little signal about the label. The selected Logistic Regression had the best cross validated macro F1 (0.284) of five tuned models, and its test macro F1 (0.222) is double the majority class baseline (0.105). That is why the design never lets the model act alone: its output is a starting suggestion, the rules and validator enforce safety, and a trainer approves every plan. Show: FitWise_Prototype_Lab/ml/artifacts/model_results.json.

Q: Why macro F1 and not accuracy? Macro F1 averages the F1 of each class equally, so a model cannot score well by always predicting the most common class. The baseline shows this: 26.7% accuracy but only 0.105 macro F1.

Q: Why choose the model on cross validation, not the test score? If you pick the model that scores best on the test set, the test set is no longer an unbiased check. GridSearchCV with 5 fold stratified cross validation picks the model and settings using training data only; the test set is used once, at the end.

Q: Why only those 8 features? They are things a brand new member can provide at the first assessment. Features in the dataset like calories burned or session duration describe past workouts, which a new member does not have yet. That is what feature_profile "new_member_safe_v1" means.

Q: What does class_weight="balanced" do? It gives more weight to classes with fewer examples during training, so the model does not simply favour the largest class.

Q: Why put preprocessing inside the Pipeline? So the scaler and encoder are fitted only on training folds during cross validation (no data leakage), and the exact same transformations are saved in the joblib file and reused at prediction time.

Security

Q: How do you stop a trainer seeing another trainer's members? Row Level Security in PostgreSQL. Every table has policies that compare the row's owner with auth.uid(). Even if the route code had a bug, the database would return no rows. Show: supabase/migrations/0002_members_assessments.sql.

Q: What is the service role key and why is it dangerous? It bypasses all RLS. It is only used in lib/supabase/admin.ts, which imports "server-only" so the build fails if any browser code imports it. It is used only for actions RLS cannot express, such as creating a login for a new member.

Q: Can a member make themselves a trainer or admin? No. The trigger protect_profile_privileges raises an error if anyone other than the server changes role, status or email. New accounts always start as member through the handle_new_user trigger.

Q: How are members created if there is no member signup? The trainer creates them. POST /api/trainer/members creates a Supabase Auth user with a random 16 character temporary password, links them to the trainer in members, and returns the password once for the trainer to hand over.

Plans, validation and workflow

Q: What happens if the validator finds problems? The plan is still saved as a draft with its score and list of violations, so the trainer can see why. Approval of an invalid plan is blocked unless the trainer gives an override reason, and the route enforces this even if someone bypasses the UI.

Q: How do you keep a record of what trainers changed? plan_audit_log stores every edit, approval and rejection with who did it, when, the reason and the previous version of the days. admin_audit_log does the same for admin actions.

Q: Is the rule based generator really deterministic? Yes. There is no randomness: exercises are sorted by category order then ID, and each day rotates the list by 3 places. The same input always gives the same plan, which makes it testable and explainable.

Q: How is the progress projection calculated? Is it a prediction? No, it is a transparent scenario. Fixed monthly rate for the goal (weight loss −2 kg, muscle gain +0.25 kg, others 0) multiplied by adherence (completed days divided by planned days) over 1, 2 and 3

months. It is not fitted to data because the dataset has no repeated measurements of the same person. Show: lib/progress.ts.

Mobile

Q: How does the Flutter app know who is logged in? supabase_flutter keeps the session on the device. AuthGate listens to onAuthStateChange and shows either the login screen or the home screen.

Q: Why does the app call Supabase directly instead of your API? RLS already protects the data, so a simple read or insert is safe from the client and avoids an extra server hop. The API is used only when server side logic is needed, which is the progress projection.

Q: Why setState and not a state management library? Each screen owns its own small amount of data, loaded once. setState keeps the code simple and readable for an app of this size; a library like Provider or Bloc would add structure that is not yet needed.

Limitations and future work

Q: What would you improve with more time?

- Train on a dataset labelled by fitness professionals so the model learns recommended, not observed, workouts.
- Collect repeated measurements of members so the progress screen can use a real forecast.
- Add automated tests for the generators and validator.
- Move secrets for the mobile app out of the bundled .env file and run the ML service on a hosted server.
- Add push notifications and trainer to member chat.

9. Cheat sheet

The numbers and names to have ready without looking anything up.

Fact	Answer
Roles	admin, trainer, member (status: pending, active, disabled)
Workout categories	Cardio, Strength, HIIT, Yoga
Dataset	Kaggle Gym Members Exercise Tracking, 973 rows, 778 train / 195 test

Fact	Answer
ML features	8: age, gender, weight, height, fat %, workout frequency, experience level, BMI
Selected model	Logistic Regression, C = 0.1, class_weight balanced
Models compared	5 tuned (LR, Decision Tree, Random Forest, SVM, Gradient Boosting) + majority baseline
Selection metric	5 fold stratified CV macro F1 (LR: 0.284)
Test results	accuracy 0.236, macro F1 0.222 (baseline macro F1 0.105)
Rule based rules	6 (RULE-01 to RULE-06)
Gemini prompt constraints	9, plus a JSON response schema
Validator bounds	sets 1 to 10, reps 1 to 50, rest 0 to 300 s, at least 2 exercises a day
Experience prescriptions	beginner 2 x 10 (75 s), intermediate 3 x 10 (60 s), advanced 4 x 8 (60 s)
Exercise catalogue	28 seeded exercises, IDs like STR-001, CAR-001, HIT-001, YOG-001
Plan statuses	draft, approved, rejected
Progress rates	weight loss −2 kg/month, muscle gain +0.25 kg/month, others 0, scaled by adherence
Migrations	11 SQL files, 0001 to 0011
Mobile tabs	Home, Plan, Workouts, Progress, Profile
Ports in development	ML API 8000; web app 3002 (the address the mobile .env points to)

Key files to be able to open and explain line by line:

1. FitWise_App/web/lib/ruleBased.ts
2. FitWise_App/web/lib/validator.ts
3. FitWise_App/web/lib/geminiGenerator.ts
4. FitWise_App/web/app/api/plans/generate/route.ts
5. FitWise_App/web/app/api/plans/[id]/approve/route.ts
6. FitWise_App/web/lib/auth.ts and lib/authApi.ts

7. FitWise_App/web/supabase/migrations/0002_members_assessments.sql (RLS example)
8. FitWise_Prototype_Lab/ml/train.py and app.py
9. FitWise_App/mobile/lib/main.dart

How to run it for a demo: start the ML API (cd FitWise_Prototype_Lab/ml, activate .venv, uvicorn app:app --port 8000), then the web app (cd FitWise_App/web, npm run dev -- -p 3002), then the Flutter app (cd FitWise_App/mobile, flutter run). Check http://localhost:3002/api/health first: both supabase and ml should show ok true.